LEGAL DOMAIN RAG · TECHNICAL REPORT

법률도메인 RAG

구조화·실험 종합보고서

대한민국 법령·금융규제 검색에서 명제별 법률 컴파일러까지

법체계 진입 · 조문 회수 · 관계 확장 · 근거 결속 · 적용 가능성 심사 · 답변

기준일 2026-08-31

실험 범위 ADR-0001 계열 ~ ADR-0121

1. 한눈에 보는 결론

이 프로젝트의 핵심 성과는 단순한 벡터 검색기를 만든 것이 아니다. 법률가가 검토할 때 사용하는 구조를 데이터와 실행 흐름으로 옮겼다는 데 있다.

```
사용자 사실·질문
  → 쟁점 분해
  → 적용 법체계와 규범 역할 선택
  → 법령 → 조 → 항·호 순서의 탐색
  → 위임·준용·정의·예외 등 연결 조문 확장
  → 질문의 필수 명제별 근거 결속
  → 적용 가능성·충돌·누락 심사
  → 근거가 허용하는 범위에서 답변
```

실험에서 반복해서 확인된 사실은 다음과 같다.

1. **회수율과 답변 정확도는 다르다.** 정답 조문이 최종 context 에 있어도 LLM 이 틀릴 수 있다.
2. **넓게 회수한 뒤 전역 top-k 로 자르는 방식은 법률 검토에 맞지 않는다.** 작은 예외·정의·위임 근거가 잘린다.
3. **법률 좌표와 시점 정합성이 먼저다.** 폐지·개정 전 조문이나 잘못된 항·호는 의미 검색으로 복구할 수 없다.
4. **법률관계 그래프는 정답을 표현하고 연관 조문을 확장하는 데 유효하지만, 질문에서 최초 진입점을 찾는 기능을 자동으로 해결하지 않는다.**
5. **답변 후 verifier 만 추가하면 늦다.** 조문 선택, 명제 생성, 근거 결속, 포섭 직전에 작은 컴파일 단계를 넣어야 한다.
6. **전역적인 답변 차단은 회귀가 크다.** 누락은 질문 전체가 아니라 해당 명제만 보류하고, 그 누락만 표적 검색해야 한다.
7. **현재 가장 유망한 구조는 회수용 스키마와 컴파일용 스키마의 이단계 구조다.** 전자는 놓치지 않는 것이 목적이고, 후자는 근거 없는 결론을 만들지 않는 것이 목적이다.

2. 법률도메인을 시스템에 어떻게 적용했는가

2.1 법률 좌표와 시점

일반 문서 RAG 의 chunk ID 대신 다음 좌표를 기본 단위로 삼았다.

```
법령 ID
  └ 조(article)
      └ 항(paragraph)
          └ 호(item)
              └ 목(subitem)
```

여기에 공포일·시행일·현행 세대·출처 hash 를 결합했다. 따라서 "제 15 조"라는 문자열이 같아도 어떤 법령의 어느 시점 조문인지 구별한다. ADR-0082~0091 은 골든 좌표 오류, 현행/과거 세대 혼재, 파서 불일치, source materialization 문제를 순서대로 정리했다.

법률적 의미는 명확하다. 기업의 행위 가능성을 판단할 때 현재 효력이 없는 조문을 근거로 삼거나, 같은 번호의 다른 법 조문을 인용하면 나머지 추론이 모두 맞아도 결론은 틀린다.

2.2 규범의 역할

모든 조문을 같은 검색 문서로 보지 않고 역할을 구분했다.

규범 역할	법률 검토에서 하는 일	검색·컴파일 동작
정의	용어의 법정 의미와 적용 범위를 고정	질문 용어와 정의조를 조기에 결속
본문 규칙	원칙적 의무·금지·허용·권한을 정함	결론 명제의 주된 근거
요건	행위가 허용되거나 의무가 발생하는 조건	사실과 하나씩 대조
예외·특례	원칙의 적용을 배제하거나 변경	본문과 원자적 bundle 로 보존
절차	승인·보고·통지·기간·방법	실무 행동계획으로 변환
제재	위반 시 효과	위험 설명과 후속 조치에 사용
위임 근거	하위 규범이 정할 수 있는 범위를 설정	상위법→하위법 수직 탐색
집행·권한 배분	누가 권한을 행사하거나 업무를 수행하는지	행위 주체와 권한 근거 검증

규범 역할	법률 검토에서 하는 일	검색·컴파일 동작
	설정	
준용·적용·예에 따름	다른 규범 내용을 현재 사안에 끌어옴	수평적 rule incorporation

이 구분 덕분에 사용자의 "이렇게 해도 되나?"를 단순 유사도 문제가 아니라 `행위 주체 + 행위 + 대상 + 조건 + 적용 규칙 + 예외 + 절차` 문제로 바꿀 수 있다.

2.3 법률관계의 세 축

문서와 Legal Graph 에서 혼동되기 쉬운 관계를 세 축으로 분리했다.

8. **규범 형성의 수직축**: 입법위임, 수임, 재위임, 위임 범위
9. **권한·업무 주체축**: 행정권한 위임·수임·재위임, 업무위탁, 대리, 내부전결, 임용
10. **규범 내용의 수평축**: 적용, 준용, 따른다, 예에 따른다, 준하여, 예외, 불구하고

첫 축은 하위 규범을 만들 권한의 근거와 한계를 찾는다. 둘째 축은 실제 행위를 누가 적법하게 할 수 있는지를 찾는다. 셋째 축은 다른 조문의 내용을 어느 정도 변형하여 가져오는지를 찾는다. 세 축을 같은 `related_to` 로 뭉치면 "연결은 찾았지만 법률효과는 틀린" 결과가 생긴다.

2.4 질문의 법률적 구조화

질문에서 다음 요소를 추출하되, LLM 이 임의 값을 만들게 두지 않는 방향으로 발전했다.

```
{
  "issue_family": "업무위탁",
  "actors": ["금융회사", "외부 기관"],
  "action": "위탁",
  "object": "심사업무",
  "conditions": [],
  "quantity": null,
  "cited_authority": null
}
```

핵심 원칙은 세 가지다.

- 사용자가 명시한 법명·조문은 가장 강한 anchor 로 취급한다.
- actor/action/object 는 DB 의 closed-set ID 후보와 결속하되, 불확실하면 검색 자체를 막지 않는다.
- `MEDIUM` 또는 `AMBIGUOUS` 후보는 AND 필터가 아니라 soft boost 나 병렬 branch 로 사용한다.

ADR-0105 의 schema-constrained grounding 은 없는 DB 개념을 만들어내는 오류를 줄였지만, 최종 검색 성능과 지연 문제 때문에 MAIN 에는 연결되지 않았다.

2.5 법률가식 계층 탐색

검색 순서를 `전체 코퍼스 → top-k` 한 번으로 끝내지 않고 다음처럼 만들었다.

```
쟁점(issue branch)
  → 어떤 법률 영역인가
  → 어떤 법령이 지배규범인가
  → 그 법령에서 어느 역할의 조문을 먼저 볼 것인가
  → 해당 조문의 인용·위임·준용·정의 관계를 어디까지 따라갈 것인가
  → 필요한 근거가 없으면 다음 법령 또는 다음 페이지로 이동
```

법령별 역할 지도와 정의조 선진입, 페이지 cursor, no-hit 후 다음 법령 이동을 도입했다. 사람처럼 순서가 있지만, 서로 독립적인 쟁점과 법령 후보는 병렬로 조사할 수 있게 설계했다.

2.6 회수용 스키마와 컴파일용 스키마

둘은 목적이 다르다.

계층	목적	실패를 다루는 방식
회수용 스키마	관련 있을 가능성이 있는 법령·조문을 놓치지 않음	fail-open, 넓은 후보, 병렬 branch
컴파일용 스키마	답변 명제가 실제 법문으로 지지되는지 확인	claim-local fail-closed, exact evidence

회수 단계에서는 full text, vector, reverse match, 명시 조문, 법체계 지도, 그래프 관계를 합친다. 컴파일 단계에서는 `명제 → 적용 규범 → exact quote → 사실 대조 → 예외/충돌 → 결론 허용 범위`를 검사한다.

2.7 Coverage 와 포섭

`coverage`는 "관련 조문을 하나 찾았는가"가 아니다. 특정 법률관계에서 판단에 필요한 항목이 채워졌는지를 뜻한다.

예를 들어 업무위탁이라면 다음이 서로 다른 slot 이다.

```
위탁 가능 근거
위탁 가능한 업무 범위
수탁자 자격
금지·제한
승인·보고·통지
예외·특례
```

기존 behavior engine 은 누락 시 `BLOCKED_MISSING_COVERAGE`, 일반 인용이면 `NAVIGATION_ONLY`, 검토 준비가 되면 `READY_FOR_ASSESSMENT`를 반환한다. 다만 이것은 "판단 자료가 충분한가"까지이고, 실제 사실을 각 요건에 포섭하여 가능·불가능·조건부 가능을 만드는 마지막 계층은 별도다.

3. 전체 실험의 진화 과정

3.1 1 기 — 검색 도달성과 LegalClosure

초기 ADR-0001~0022 의 목표는 정답 조문을 후보에 넣는 것이었다. lexical, vector, query expansion, 인용관계, 위임관계, 정의 연결을 조합하여 단일 검색보다 높은 recall 을 만들었다.

이 시기에 얻은 교훈은 다음과 같다.

- 벡터만으로는 정확한 숫자·법률용어·후반부 항·호를 놓친다.

- lexical 만으로는 사용자의 생활어와 법정용어 사이를 건너지 못한다.
- 인용·위임 관계를 따라가면 recall 은 오르지만 관련성이 낮은 조문도 폭증한다.
- 따라서 최초 회수와 관계 확장, 최종 제시는 서로 다른 단계여야 한다.

과거 70 문항 계열에서 LegalClosure v2 는 높은 조문 도달성(69/70)을 보였지만, 답변 정확도는 별개의 문제로 남았다. 이 수치는 후대 Golden 61 과 데이터셋이 달라 직접 비교하지 않는다.

3.2 2 기 — 정의·관계·법률 프레임

ADR-0023~0042 에서는 정의조, 위임·수임·재위임, 준용·적용, 법률 효과 frame 과 rule executor 를 만들었다.

대표적인 실패가 ADR-0040 의 frame-only 답변이다. 원문 조문을 충분히 주지 않고 구조화 frame 만 제공하자 정답이 `32→4` 로 붕괴했다. 구조는 원문을 대체할 수 없고, 원문을 선택하고 설명하는 보조 계층이어야 한다는 강한 반증이었다.

정의 카드를 무조건 inline 한 실험도 `26→23` 으로 하락했다. 정의는 중요하지만 모든 질문에 모든 정의를 넣으면 오히려 핵심 규칙을 희석한다. 이후 정의는 `질문 용어가 실제로 그 정의에 의존할 때` 조기 진입하는 방식으로 바뀌었다.

3.3 3 기 — 실험 거버넌스와 인과 분리

ADR-0043~0060 에서는 결과 자체만큼 실험 방법을 고쳤다.

- ADR 번호와 scope 를 원자적으로 예약
- dataset·prompt·모델·judge·입력 hash 동결
- product model 과 offline evaluator 분리
- 모델 상한, prompt 효과, 추가 조문 간섭을 별도 arm 으로 측정
- MAIN 승격을 Accepted OOS arm 과 명시적 receipt 로 제한

질문만 또는 gold article 만 주는 ceiling 실험은 검색과 추론을 분리하는 데 사용됐다. 예를 들어 51 문항 계열에서 Haiku 상한은 약 `45/51`, 더 강한 모델의 최고 관찰치는 `49/51` 이었다. 즉 완벽한 조문을 주어도 답변 오류가 남으므로, 검색 개선만으로 100%에 도달할 수 없다는 뜻이다.

ADR-0100 의 Opus 질문-only 61 은 strict consensus `33/61`, 한 judge 이상 정답 `40/61` 이었다. 법령 근거 없이 모델의 기억만 사용하는 방식은 세부 요건·특례에서 한계가 분명했다.

3.4 4 기 — 후보 생성과 선택 손실 분리

ADR-0061~0081 은 "못 찾은 것"과 "찾았지만 버린 것"을 분리했다.

주요 장치는 다음과 같다.

- raw pool, closure 후, selector 후, final present 의 단계별 원장
- 정답 좌표가 어느 단계에서 생기고 사라졌는지 기록
- proof span 과 source provenance 검증
- baseline add-only 보호
- bounded union 과 cap 실험

raw pool 을 넓혀도 최종 top-20 개선은 작았다. 한 실험에서는 raw recall 이 `64→68/83` 으로 올랐지만 top-20 은 `64→65` 에 그쳤다. 생성보다 selection 이 병목이라는 증거다.

이 시기에 전역 상위 N 개 보호가 충분하지 않다는 것도 드러났다. 질문에 여러 쟁점이 있으면 점수가 높은 한 branch 가 슬롯을 독점한다. 이후 `(issue branch, legal role)` 별 quota 와 claim-evidence 원자적 cap 으로 발전했다.

3.5 5 기 — 데이터·좌표 정본화

ADR-0082~0091 은 검색 알고리즘보다 데이터 정합성을 먼저 고쳤다.

- Golden corpus 자체의 현행성 감사
- 시행일·법령 generation 관리

- 공통 legal coordinate parser
- 법령 원문 좌표 materialization
- corrected Golden 61 재채점

ADR-0091 의 동결 기준에서 decisive exact hit 는 `71/90`, 모든 decisive 를 맞힌 질문은 `44/61` 이었다. 미회수 19 좌표 중 일부는 후보가 생성됐다가 마지막 단계에서 버려졌고, 일부는 최초 후보에 전혀 생기지 않았다. 이 구분이 이후 구조의 출발점이 됐다.

3.6 6 기 — 관계 기반 표적 회복

ADR-0092~0096 은 잔여 miss 를 일반 검색이 아니라 법률 역할과 predicate 로 복구하려 했다.

실험	결과	해석
bounded companion hop	위임 관련 일부 `1/4→2/4`	2-hop 은 유효하나 폭·지연이 큼
late seed/delegation audit	정답 +1, raw 기회 59 개	precision 약 1.69%로 너무 낮음
role-predicate governing router	`71/90→73/90`, 기존 손실 0	표적 branch 는 유효하나 in-sample
feedback API helper	안전 계약 통과	발행 답변은 바꾸지 않는 학습 신호 수집
remaining miss replay	`12/29→12/29`	predicate gate 전에 카드가 탈락하여 복구 0

중요한 교훈은 관계 확장 자체가 아니라 **언제, 어떤 법률 역할에서, 어떤 predicate 로 확장하는가**가 성능을 결정한다는 것이다.

3.7 7 기 — Legal Graph 와 query family

ADR-0097~0104 는 법률 지식을 sidecar graph 로 만들고 실행 계약을 부여했다.

- Legal Graph v1: node, relation, source span, authority generation, confidence
- extensible query-family registry
- REQUIRED relation 의 bounded 1-hop closure
- invalid/stale graph 의 fail-open 처리

- 관계별 coverage 와 금지 추론을 정의한 behavior runtime

ADR-0099 는 graph closure 로 decisive 를 `71/90→72/90` 으로 1 개 올리고 기존 좌표 손실은 0 으로 유지했다. ADR-0101 hardening 도 `72/90` 을 보존했다. 그래프는 작지만 안전한 순증을 만들었다.

반대로 ADR-0103 의 semantic entity pipeline 은 잔여 29 좌표에서 `13/29→15/29` 에 그쳐 목표 `20/29` 를 넘지 못했고, p95 도 기준을 초과했다. typed entity 가 존재해도 다중 규칙 공동선택과 최종 bundle 보존을 해결하지 못했기 때문이다.

ADR-0104 behavior runtime 은 관계 16 개에 coverage 와 결론 gate 를 연결했다. 수임을 위임과 동일시하거나, 위탁 사실만으로 권한 전체 이전을 추론하거나, 준용을 원문과 완전히 동일하다고 보는 과잉 추론을 금지했다. 다만 독립 엔진으로 MAIN 에는 연결하지 않았다.

3.8 8 기 — 사용자 표현 grounding 과 검색 trace

ADR-0105 는 다음 흐름을 프로토타입으로 구현했다.

```
사용자 원문 span
  → reverse/full-text/vector 후보
  → DB 에 존재하는 ID 만 closed menu 로 제시
  → LLM 선택
  → signature 검증
  → unresolved 허용
```

잔여 29 좌표 평가에서 기존 `15/29`, grounding 적용 `17/29`, 중간 typed closure 최고 `21/29` 였지만 최종 rerank 뒤 다시 `17/29` 로 내려갔다. p95 는 약 33 초였다. 이는 grounding 이 쓸모없다는 뜻이 아니라, **중간에 회복한 정답을 final selector 가 제거했다**는 뜻이다.

ADR-0106 의 threshold trace 는 더 구체적인 병목을 보였다. `100 분의 10`, `주관회사`, `계열회사 보유분` 질문에서 raw 후보는 많았지만 정답 조문 순위가 final cap 밖이었다. 당시 조문 vector 가 본문 앞 800 자만 사용하여, 2 천~3 천 자 뒤의 핵심 숫자와 용어가 embedding 입력에 아예 없었던 것도 원인이었다. 숫자 정규화가 있어도, 그 숫자가 포함된 span 을 후보 생성기에 넣지 않으면 도움이 되지 않는다.

3.9 9 기 — 법체계 지도와 계층형 조사 controller

ADR-0107~0113 은 변호사식 탐색을 구현하려 한 계열이다.

목표 구조는 다음과 같았다.

```
질문
  → issue branch
  → 적용 법령 후보와 역할
  → 법령별 article page
  → no-hit 이면 cursor 를 보존하고 다음 page/법령
  → definition·exception·delegation companion 조사
```

초기 실험은 검색 논리보다 structured-output provider 계약에서 실패했다.

- ADR-0108: numeric schema 관련 provider 오류로 11/11 실패
- ADR-0109: `uniqueItems` 등 schema 호환 문제로 11/11 실패
- ADR-0110: 공식 transform 으로 HTTP 400 은 제거했지만 local atomic wave 검증 실패, candidate `0/36`
- ADR-0111: provider-neutral contract 를 도입하여 Anthropic Haiku 와 onsite Qwen 의 capability projection 을 분리하고 6/6 구조 기준·61/61 regression 통과
- ADR-0112: structured call `193/193` 은 유효해졌지만 탐색 예산 내 정답 회수는 `1/36`, 최종 상태는 `INCOMPLETE_WITH_CURSOR`

이 계열의 가장 중요한 성과는 "LLM 이 JSON 을 잘 냈다"가 검색 성공이 아니라는 사실을 분리한 것이다. provider contract, local canonical validation, page semantics, cursor, actual retrieval recall 을 각각 독립 검증해야 한다.

ADR-0113 은 법령·의미 진입점을 LLM 선택과 별도로 보존하는 full-stack entry routing 을 만들었다. 초기 진입점이 틀려도 lexical/semantic anchor 를 같이 살려 두는 이중 경로가 이후 통합 상태 머신의 기반이 됐다.

3.10 10 기 — 통합 조사 상태 머신

ADR-0114 는 법체계 지도, grounding, 관계 closure, role quota, coverage compilation 을 한 흐름에 합쳤다.

지표	baseline	통합 7 계층
답변 정답	22/61	36/61
decisive 좌표	72/90	76/90
필수 companion	13/22	15/22

검색과 답변은 크게 개선됐지만 `unsupported_definitive_conclusion` 이 21 건 발생하여 Rejected 됐다. 관련 조문을 더 잘 찾았어도, 그 조문이 질문의 구체적 결론을 실제로 지지하는지 확인하는 포섭 계층이 부족했다.

이 실험으로 구조의 중심이 검색에서 컴파일러로 이동했다.

3.11 11 기 — 법률 컴파일러의 반복 실험

ADR-0115: coverage-preserving compiler

- 정답 `29/61→35/61`
- 입력 토큰 약 +34.1%
- baseline-only 회귀 9 건
- unsupported `20→25` 로 악화
- decisive `78/90→76/90`, companion `16/22→15/22`

평균 evidence 를 늘리고 coverage 를 표시했지만, 선택기가 확인된 decisive 근거까지 제거하고 gate 가 답할 수 있는 질문도 과도하게 막았다.

ADR-0116: scope·entailment·bundle

- paired rerun `23/61→35/61`
- decisive `77/90→80/90`

- unsupported `24→17`
- 그러나 답변 회귀 8 건, companion 1 개 감소, claim/gate 위반 7 건

법령·시장 scope, exact quote, bundle 보호는 효과가 있었다. 그러나 baseline 자체 변동이 크고 claim 과 evidence 가 서로 다른 cap 에서 분리되는 문제가 남았다.

ADR-0117: issue-role atomic compiler

- opaque closed ID 를 사용했으나 40/41 사례에서 gate coercion 발생
- verified gold 는 많이 보존했지만 실제 compiler unit 이 부족
- 정답이 오히려 `24→21` 로 하락

ID 를 닫는 것만으로는 충분하지 않았다. LLM 이 ID 의 법률 의미를 이해할 수 없거나, coverage 불완전을 질문 전체의 답변 거부로 바꾸면 안전해 보이지만 유용성은 급락한다.

ADR-0118: integrated atomic claim compiler

- rerun 기준 `26/61→38/61`, +12
- baseline-only 회귀 8 건
- 토큰 약 +66.7%, 제품 p95 약 +35.2 초

명제와 evidence 를 원자적으로 묶는 방향은 가장 큰 순증 중 하나를 만들었다. 다만 모든 답을 새로 생성하는 구조라서 이미 맞은 baseline 답변까지 다시 흔들었고 비용도 증가했다.

ADR-0119: proposition applicability compiler

`SUPPORTED / CONDITIONAL / MISSING / CONFLICTED` 상태, exact quote, closed proposition/evidence ID, 적용 가능성 검사를 독립 구조로 만들었다. 26/26 구조 계약은 통과했으나 답변 성능 실험은 아니었다.

ADR-0120: proposition assessment shadow

baseline 답변에 대한 명제별 shadow assessment 를 시도했다. fresh rerun 에서 baseline `43/61` 이 candidate `11/61` 로 붕괴했다.

- missing coverage 172 개

- retry 60/61
- answerable 4 개
- missing-direct 33 개
- unsupported `14→46`

실패 원인은 명확하다. `MISSING` 하나가 질문 전체 답변을 차단했고, 감사용 shadow 상태가 답변 생성기를 지배했다. 안전성 계층이 전역 hard gate 가 되면 false block 이 폭증한다.

ADR-0121: claim-wise non-destructive compiler

이번 개선은 위 실패를 직접 겨냥한다.

- baseline 답변을 immutable floor 로 유지
- 답변의 정확한 claim span 별로만 감사
- `MISSING` 은 해당 명제의 표적 재검색 신호일 뿐, 전역 차단 금지
- `CONDITIONAL` 또는 `CONFLICTED` 이고 exact quote·closed ID 검증을 통과한 경우에만 해당 span 수정
- 수정하지 않은 문자열은 기계적으로 동일하게 보존
- 검색 evidence 와 decisive/companion recall 은 부모 arm 과 동일하게 고정

61 문항 결과는 `18/61→18/61` 로 순증 0 이었다. 검색·evidence 는 decisive `77/90`, companion `14/22`, 총 1,708 개로 완전히 동일했고 전역 block 도 0 이었다. 그러나 157 개 claim 중 MISSING 이 102 개였고, answer exact-span 위반 28 개, exact-quote 위반 24 개, replacement-ground 위반 3 개가 발생했다. 안전 검증을 통과한 실제 patch 는 5 문항 6 개뿐이며 정오를 바꾼 것은 없었다. 따라서 `Rejected` 다.

이 결과는 국소 비파괴 정책은 옳지만 **답변을 먼저 자유 생성한 뒤 그 문자열에서 claim 을 재발견하는 순서**가 잘못됐음을 보여준다. 다음 compiler 는 proposition ID 와 source span 을 먼저 고정하고 그 단위로 문장을 생성해야 한다.

또한 ADR-0120 에서 `43/61` 로 평가됐던 동일 봉인 baseline 답변이 이번 fresh paired Haiku 에서는 `18/61` 로 평가됐다. 후보 답변과 함께 한 번에 채점하는 방식과 단일 sampling 의 변동이 큰 것이다.

따라서 run 내부 paired delta 는 사용할 수 있지만 서로 다른 run 의 절대점수를 단일 리더보드처럼 비교해서는 안 된다. 후속 평가는 독립 arm grading 또는 고정 다회 consensus 와 결정론적 법률 검사를 병행해야 한다.

4. 무엇이 왜 실패했는가

4.1 "더 많이 넣기"의 실패

후보와 조문을 많이 넣으면 recall 은 오르지만 다음 문제가 생겼다.

- 중요한 예외와 정의가 긴 context 에 묻힘
- 서로 다른 법령·시장·시점 규칙이 혼합됨
- LLM 이 일반 규칙을 구체적 특례보다 우선함
- token·latency 가 증가하지만 정답 증가는 작음

따라서 후보 수의 전역 cap 보다 쟁점·법률 역할별 최소/최대 quota 가 적합하다.

4.2 "AND 검색"의 실패 가능성

actor, action, object 를 정확히 뽑았다면 AND 는 강력하다. 그러나 하나만 잘못 grounding 하면 정답 집합이 0 이 된다. 그러므로:

- 사용자 명시 법명·조문·정확 숫자: hard anchor 가능
- DB closed-set 에 정확히 결속된 고신뢰 용어: 제한적 AND
- 일반 actor/action/object: soft boost
- ambiguous: 후보별 병렬 검색

으로 신뢰도에 따라 연산자를 달리해야 한다.

4.3 "그래프가 있으면 찾는다"의 실패

Legal Graph 의 semantic card 가 정답 조문의 의미와 관계를 잘 표현해도, 질문에서 그 node 에 닿는 entry index 가 없으면 검색은 시작되지 않는다. 즉:

> `그래프 표현 가능성 ≠ 질문에서의 도달 가능성`

그래프는 최초 검색기이기도 해야 하며, 동시에 raw 검색이 찾아온 좌표를 설명하고 closure 하는 도구여야 한다. entry synonym, role/predicate index, actor/action/object grounding 이 이 간극을 메운다.

4.4 "전역 rerank 와 cap"의 실패

중간 typed closure 가 `21/29` 까지 정답을 포함했는데 final 이 `17/29` 로 내려간 사례처럼, 정답을 찾고도 버리는 손실이 컸다. 원인은 전역 점수로 서로 다른 역할의 조문을 경쟁시킨 것이다.

개선 원칙은 다음과 같다.

- governing rule, definition, exception, authority basis 를 서로 다른 quota 로 보호
- claim 과 그것을 지지하는 evidence 를 하나의 원자 단위로 cap
- 한 번 검증된 baseline decisive evidence 는 후속 rerank 가 제거하지 못하게 함

4.5 "coverage 부족이면 전체 차단"의 실패

법률 검토에서 자료가 하나 부족하다고 모든 문장을 말할 수 없는 것은 아니다. 예를 들어 위탁 가능 근거는 명확하지만 신고 요건이 발견되지 않았다면:

- "위탁 자체의 법적 근거"는 답할 수 있음
- "신고가 불필요하다"는 답하면 안 됨
- 신고 요건만 표적 검색해야 함

ADR-0120 은 이를 전역 차단으로 처리해 붕괴했다. 올바른 단위는 질문이 아니라 명제다.

4.6 "구조화 출력 성공 = 법률 검색 성공"의 실패

schema 가 provider 에서 거절되거나 sentinel 이 실제 법률 ID 처럼 노출되는 문제도 있었지만, 이를 고친 뒤에도 검색 recall 은 낮았다. 구조화 출력은 실행 인프라의 필요조건일 뿐, 법률적으로 올바른 법령·조문을 고르는 충분조건이 아니다.

4.7 "답변 후 검증"의 한계

완성된 답을 마지막에 검증하면 오류 위치는 알 수 있어도 다음 문제가 남는다.

- 이미 핵심 근거가 context 에서 빠졌음
- 잘못된 규칙으로 답 전체가 구성됨
- verifier 가 고치기 위해 답을 다시 생성하면서 맞은 문장까지 회귀

그래서 컴파일은 한 번의 마지막 검사라기보다 핵심 구간에 반복 삽입되어야 한다.

> 회수 후: 필요한 법률 역할이 있는가
> 선택 후: `claim-evidence bundle` 이 보존됐는가
> 포섭 전: 해당 법문이 이 사실관계에 적용 가능한가
> 답변 후: 인용과 결론 범위가 일치하는가

5. 지금까지 효과가 검증된 자산

5.1 데이터 계층

- 현행 authority generation 과 effective date
- 공통 legal coordinate parser
- source span 과 hash provenance
- corrected Golden 61 과 decisive 90·companion 22 원장
- 법령·규범 역할 지도

5.2 검색 계층

- lexical/vector/reverse-match 의 fail-open union
- 명시 법명·조문 direct anchor
- query-family 조건부 graph closure
- role-predicate 표적 branch
- 단계별 recall/discard ledger
- baseline add-only 보호

5.3 법률 그래프·행동 계층

- 위임·권한 배분·업무위탁·준용 관계 분리
- 관계별 필수 coverage
- 금지되는 과잉 추론
- `NAVIGATION_ONLY / BLOCKED_MISSING_COVERAGE / READY_FOR_ASSESSMENT`
- provider-neutral closed-menu 와 local canonical validation

5.4 컴파일 계층

- exact quote 와 closed evidence ID
- proposition 단위 `SUPPORTED / CONDITIONAL / MISSING / CONFLICTED`
- issue-role 별 evidence quota
- claim-evidence 원자적 bundle
- baseline answer 의 비파괴적 보존 원칙

6. 권장 최종 구조

```
[A. 질문 분석]
  원문 span, 명시 법명·조문·숫자 보존
  issue branch 생성
  actor/action/object 는 closed-set 후보 + confidence

[B. 법체계 진입]
  issue 별 governing-law 후보
  정의/본문/예외/절차/권한 역할별 진입점
  고신뢰 branch 병렬 probe

[C. 넓은 회수]
  lexical + vector + reverse match + direct coordinate
  실패해도 다른 채널 유지
  조문 전체가 아니라 근처 text + 정확 좌표 반환

[D. 법률관계 확장]
  위임·준용·정의·예외·참조 edge
  질문 family 에 필요한 relation 만 bounded closure

[E. 역할별 선택]
  (issue branch, legal role) quota
  governing/definition/exception/authority 최소 슬롯
  검증된 baseline evidence 보호

[F. 명제 컴파일]
  질문의 필수 명제 목록
  claim-evidence 원자 결속
  exact quote·authority·scope·time·applicability 검사
  MISSING 은 해당 claim 의 targeted retry

[G. 답변]
  지지된 명제는 답함
  조건부 명제는 조건을 명시
  충돌은 충돌 규범과 우선순위를 설명
  누락 명제만 유보
  인용은 정확 좌표와 함께 제시
```

```
[H. 원장·평가]
  각 단계 후보 생성/제거 이유
  provider raw receipt
  retrieval recall, answer correctness, unsupported 를 분리 채점
```

핵심 정책은 `검색은 fail-open, 법률 결론은 claim-local fail-closed` 다.

7. 데이터 보강과 구조 보강의 우선순위

둘 중 하나를 전부 끝낸 뒤 다른 하나를 시작하는 방식은 적합하지 않다. 다음 순서가 비용 대비 효과가 높다.

P0 — 현재 컴파일 경계 안정화

11. ADR-0121 에서 확인된 span·quote·replacement 위반을 proposition-first 구조로 제거
12. baseline-only 0 과 global block 0 이라는 비파괴 하한은 유지
13. 동일 답변의 judge 변동을 독립 arm 또는 고정 다회 consensus 로 통제
14. claim-evidence 원자성 위반을 0 으로 고정

P1 — 실패 문항 기반 데이터 보강

15. never-generated 정답 좌표에 entry synonym·법률 역할·predicate 보강
16. 긴 조문을 항·호 또는 의미 span 단위로 재임베딩
17. 숫자·비율·기간을 원문 span 과 함께 별도 index
18. 정의·예외·위임 근거의 REQUIRED companion edge 보강
19. 52.4%에 머문 lexicon vector cache 완성

P2 — 표적 구조 개선

20. issue-role quota 를 실제 selector 의 cap 에 일관되게 적용
21. 중간 검증 정답의 protected lineage

22. 누락 coverage 별 targeted search template
23. law/page cursor 를 실제 검색 API 와 연결
24. Anthropic Haiku 와 onsite Qwen 이 같은 canonical local validator 를 사용

P3 — 포섭과 최종 답변

25. 규범의 요건을 명제 단위로 분해
26. 사용자 사실과 요건을 충족/불충족/불명으로 결속
27. 예외·특례·충돌 우선순위 적용
28. 가능/불가능/조건부 가능과 필요한 후속 행위를 생성
29. OOS holdout 과 실제 변호사 검토로 평가

데이터 보강은 많이 추가가 아니라 stage ledger 상 정답이 한 번도 생성되지 않은 위치를 중심으로 해야 한다. 구조 보강은 이미 생성된 정답이 사라지는 단계에만 적용해야 한다.

8. 평가 지표를 어떻게 봐야 하는가

최종 정답률 하나만 보면 원인을 알 수 없다. 최소한 다음을 따로 본다.

계층	핵심 지표
데이터	현행 좌표 정확도, source span 검증률
후보 생성	decisive raw recall, never-generated 수
관계 확장	신규 정답/추가 후보 precision
선택	generated-then-dropped 수, branch/role 별 보존율
coverage	필수 slot 충족률, false missing 률
컴파일	claim-evidence 결속률, unsupported conclusion
답변	correct/61, baseline-only, candidate-only
운영	provider 오류, p50/p95, 입력 토큰, retry

특히 `정답 조문 recall`, `모든 필수 조문이 있는 질문 수`, `최종 답변 정답`, `근거 없는 단정`은 서로 대체할 수 없는 지표다.

9. 최종 판단

현재 구조는 법률 검토 로직에 가까워졌다. 법령을 무작위로 많이 가져와 cutoff 하는 구조가 아니라, 쟁점별로 법체계에 진입하고 필요한 규범 역할과 연결관계를 따라가며, 명제별로 근거를 확인하는 구조다.

다만 아직 완성된 법률판단 시스템은 아니다.

- 검색 진입점이 부족한 질문은 여전히 정답 node 에 닿지 못한다.
- 정답 조문이 있어도 LLM 의 포섭 오류가 남는다.
- coverage 추출 오류가 false block 으로 번질 수 있다.
- 컴파일러가 답 전체를 재작성하면 baseline 정답이 흔들린다.
- 반복 사용한 Golden 61 은 in-sample 이므로 MAIN 승격 근거가 될 수 없다.

따라서 다음 한 문장이 현재 방향을 가장 정확히 요약한다.

> **법률지식을 더 많이 넣는 것이 아니라, 질문이 요구하는 법률 명제마다 올바른 현행 규범과 정확한 문구를 찾아 끝까지 보존하고, 그 명제에 필요한 만큼만 결론을 허용하는 시스템으로 발전시켜야 한다.**

부록 A. ADR 계열별 지도

계열	주제	대표 ADR
초기 회수	lexical/vector/closure/relation	0001~0022
법률 의미 구조	정의, 위임, 준용, frame, executor	0023~0042

계열	주제	대표 ADR
실험 표준	lineage, evaluator, model/prompt 분리	0043~0060
reach vs selection	proof span, union, cap, stage ledger	0061~0081
데이터 정본	시점, 좌표, materialization, Golden 교정	0082~0091
표적 회복	companion hop, role-predicate feedback	0092~0096
Legal Graph	schema, family runtime, closure, behavior	0097~0104
grounding/trace	closed-set grounding, threshold 추적	0105~0106
계층형 조사	법체계 entry, page/cursor, provider contract	0107~0113
통합/컴파일	state machine, coverage, entailment, proposition	0114~0121

세부 원본과 실행 영수증은 실험 그래프, 회고, 각 ADR 의 `README.md`, `manifest.json`, `artifacts/`를 정본으로 한다.

부록 B. 용어 정리

용어	이 프로젝트에서의 뜻
canonical	동일 법률 개체를 가리키는 정본 ID·표현
axiom	관계에서 허용·금지되는 추론 규칙; 데이터 이름 전체를 대신하는 말은 아님
predicate	위임한다, 준용한다처럼 관계의 법률적 종류
query family	질문이 요구하는 법률 문제 유형
governing law	해당 쟁점을 직접 지배하는 우선 법령
companion	본문만으로 판단할 수 없어 같이 필요한 정의·예외·위임·절차 조문
coverage	결론에 필요한 법률요소가 근거로 채워진 정도
entailment	인용 문구가 특정 답변 명제를 실제로 지지하는지 여부
applicability	그 규범이 질문의 주체·행위·대상·시점에 적용되는지 여부
compiler	회수된 근거를 명제·적용범위·결론 허용 상태로 변환하는 중간 계층

용어	이 프로젝트에서의 뜻
immutable floor	이미 맞은 baseline 결과를 후속 실험이 임의로 제거하지 못하게 하는 하한